

Max Heap Construction (Bottom-Up Heapify)

Max Heap from the array using the bottom-up heapify approach.

Input Array

```
Index : 0 1 2 3 4 5
Value : 3 9 2 1 4 5
```

Number of elements:

```
n = 6
```

Last non-leaf node:

```
n/2 - 1 = 6/2 - 1 = 2
```

We perform heapify from index **2** → **1** → **0**.

Step 1: Heapify at Index 2

Current array:

```
3 9 2 1 4 5
```

- Parent = index **2** → value = **2**
- Left child = $2 \times 2 + 1 = 5$ → value = **5**
- Right child = $2 \times 2 + 2 = 6$ → Doesn't exist

Comparison:

```
5 > 2
```

Swap parent and left child:

3 9 5 1 4 2

Step 2: Heapify at Index 1

Current array:

3 9 5 1 4 2

- Parent = index **1** → value = **9**
- Left child = index **3** → value = **1**
- Right child = index **4** → value = **4**

Since **9** is already the largest, **no swap** is required.

Array remains:

3 9 5 1 4 2

Step 3: Heapify at Index 0

Current array:

3 9 5 1 4 2

- Parent = index **0** → value = **3**
- Left child = index **1** → value = **9**
- Right child = index **2** → value = **5**

Largest value = **9**

Swap parent and largest child:

9 3 5 1 4 2

Now heapify at **index 1**.

Step 4: Heapify at Index 1

Current array:

9 3 5 1 4 2

- Parent = **3**
- Left child = index **3** $\rightarrow$ **1**
- Right child = index **4** $\rightarrow$ **4**

Largest value = **4**

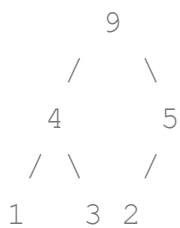
Swap:

9 4 5 1 3 2

Heapify index **4**.

Index **4** has no children, so stop.

Final Max Heap



Final Array Representation

9 4 5 1 3 2

Summary of Heapify Operations

1. Heapify(2):

3 9 2 1 4 5 → 3 9 5 1 4 2

2. Heapify(1):

No change

3. Heapify(0):

3 9 5 1 4 2 → 9 3 5 1 4 2

4. Heapify(1):

9 3 5 1 4 2 → 9 4 5 1 3 2